Room Config Builder

Operation guide for the Deployment Configurator

Building rooms, drawing them, pricing them — and teaching the app your own rules.

What is in here

Start here

What this program is for

You have a room. It has a projector, a couple of cameras, a DSP, a switcher and a touch panel, and somewhere on the processor there is a `config.json` that ties all of it together. Editing that file by hand works right up until the moment it doesn't: a key gets misspelled, a device block is left behind after the room shrank, and the room comes up wrong on a Monday morning.

The Room Config Builder is the tool that stands between you and that file. You work through tabs that show each setting with a proper label, a description and the right kind of control, and the app writes the JSON. When the room is finished you can push it straight to the processor over SFTP.

But it does more than edit the config now. The same room description drives:

- a **control schematic** — what talks to the processor, and how
- an **AV signal flow** — what plugs into what, drawn from the switcher numbers you already typed
- **rack elevations**, **floor plans** and a **cable schedule**
- a **cost estimate**, built from a device catalog you keep yourself

Most of that draws itself. You describe the room once, on the tabs you were going to fill in anyway, and the drawings and the numbers follow.

The idea worth knowing

The app tries very hard not to have opinions baked into it.

Almost everything it knows about *your* rooms lives in plain JSON files that sit next to the program, and every one of them now has an editor inside the app:

File	What it decides	Where you edit it
`ui_schema.json`	How each config key looks on screen: label, description, dropdown or switch, which device families exist at all	**Schema** tab
`av_flow_rules.json`	How a room turns into a drawing: which box a config key means, what goes between two ends that don't match, what hangs off the USB switcher	**Flow Rules** tab
`av_devices.json`	The equipment catalog: connectors, rack units, power draw, price	**Catalog** tab

File	What it decides	Where you edit it
`key_map.json`	How old files are translated to current key names on load	Text editor (reference in this guide)
`labor_rates.json`, `base_costs.json`	What an hour costs, and the rate card the estimate falls back to	Dialogs on the **Cost** tab

If the app does something you don't want — puts the wrong receiver in a run, calls a field by a name your team doesn't use, misses a key you just added to the template — the fix is usually a rule or a schema entry, not a phone call and a wait for the next build.

> Nothing in this guide requires you to edit JSON by hand. Everything described here can be done from a tab in the app. The reference chapters at the back are for when you would rather see the file.

The tabs

They run down the left-hand rail in roughly the order you use them.

Tab	What it's for
Wizard	The room's identity — building, room number, name — and how many of each device it has.
Devices	One sub-tab per device: model, IP, connection type, module, and whatever else the schema says that family has.
System	Everything else in SYSTEM_SETUP: switcher inputs and outputs, panel behaviour, outlet names.
Raw JSON	The whole config as text, for when you want to see or paste it. Apply writes it straight back to the working file.
Schematic	The control topology: what the processor talks to, over what, drawn automatically.
AV Flow	The signal flow: what plugs into what, also drawn automatically.
Floor Plan	Where things physically are, with callouts on a sheet.
Cabling	Low-voltage runs that aren't signal — screen triggers, shades, and a run schedule.
Racks	Rack elevations, front and rear, with clearances and a parts list.
Cost	The estimate: equipment, cable, labor, tax and fees.
Catalog	The device catalog every drawing and every price is built from.

Tab	What it's for
Schema	What the Devices and System tabs look like — labels, types, descriptions, device families.
Flow Rules	How the AV Flow decides what to draw.
App Config	Where the files live, the theme, the SFTP settings and the active deployment target.

The **Catalog**, **Schema** and **Flow Rules** tabs work with no room open at all — they're about the app, not about one room.

Setting up, once

Point it at a folder

The quickest setup is one folder holding everything the app reads. On first launch you'll be asked where that is; you can change it any time from **App Config**.

Every other path defaults to that Root Folder, so a folder that looks like this needs no other configuration:

```
Z:\AV\RoomConfigBuilder\
    config.json            the template a new room starts from
    processors.json        the rooms you can deploy to (name + IP)
    buildings.json         building names and their codes
    ui_schema.json         how config keys appear on screen
    key_map.json           translation of legacy key names
    av_devices.json        the equipment catalog and price list
    av_flow_rules.json     how a room draws itself
    labor_rates.json       what an hour costs
    base_costs.json        fallback prices by category
    devices\               the Python control modules
    documentation\         PDF manuals, shown in the app
```

Any file that isn't there simply isn't used: with no `av_flow_rules.json` the app draws with its built-in rules, with no `key_map.json` nothing is translated, and so on. Nothing breaks because a file is missing.

> **Put the shared ones on a share.** `av_devices.json`, `ui_schema.json` and `av_flow_rules.json` describe how your shop works, not how your laptop is set up. One copy on a network drive means two engineers drawing the same room draw it the same way. The catalog even merges another engineer's edits rather than overwriting them.

Your settings

App Config also holds the things that are yours rather than the department's: the theme (Classic or the Auris HUD look), text size, SFTP username and port, and the **Active Deployment Target** — the room the next upload or download talks to.

That target now clears itself whenever you open or create a different config. It's the quietest mistake this app could make — BSS 103's config uploaded to SSC 210's processor, with the tab that says so two screens away — so you're asked to pick the room again rather than inheriting the last one. Downloading from a processor keeps the target, because that config *came* from there.

A room, start to finish

This is the whole workflow in one page. Every step has a chapter of its own later.

1. **Start the room.** *New Config* builds from your template with every device count at zero. *Open Existing Config* loads a file from disk. *Download from Processor* pulls `/config.json` over SFTP and asks where to keep the working copy. The New Room dialog asks **Start from the cost estimator** first, because that's how a room actually starts: pick the devices out of the catalog with quantities and a running total, and they become the boxes on the signal flow, the gear in the racks and the lines on the estimate.
2. **Say what the room is.** On the **Wizard**, pick the building and type the room number, then set the device counts. Blocks and tabs appear and disappear as you change them.
3. **Fill in the devices.** On **Devices**, each one gets its model, IP or COM port, and control module. *Check Defaults* tells you what a block is missing; *Check Module Defaults* tells you where it disagrees with the driver.
4. **Fill in the system.** On **System**, the switcher input and output numbers, the panel options, the outlet names. This is the part the drawings read.
5. **Look at the drawings. Schematic** and **AV Flow** have already drawn themselves from what you typed. Fix anything that looks wrong — by dragging, or by correcting the config, or by editing a rule.
6. **Rack it and place it. Racks** for the elevations, **Floor Plan** for where things are in the room.
7. **Price it. Cost** builds the estimate from the drawing and the catalog.
8. **Hand it over.** *Save All* writes every drawing, report and workbook into one folder. *Upload to Processor* pushes the config itself.

What gets written where

The config is one file. Everything else the app knows about the room lives in small companion files beside it, so the cost estimate and the rack elevation aren't fighting over the same document:

```
BSS103_config.json                    the config itself
BSS103_config_av_flow.json            the signal flow drawing
BSS103_config_racks.json              rack elevations
BSS103_config_floor_plans.json        floor plan sheets and locations
BSS103_config_cabling.json            low-voltage runs
BSS103_config_cost.json               the estimate
BSS103_config_control_schematic.json  the control schematic layout
BSS103_old_config.json                the pristine original, from the first load
```

Opening, converting and saving

What happens when a file loads

Old configs don't look like new ones. Rather than making you fix them by hand, the app runs every load through the same pipeline, in this order:

1. **Key mapping** — `key_map.json` renames legacy sections and properties (`CAMERA1DEVICE.IPADDRESS` becomes `CAMERADEVICE_1.ip_address`). It runs first so everything after it sees current names.
2. **Automatic backup** — the original text, untouched, is written next to the working file as `<ROOM>_old_config.json`. The name comes from the room identity, which is why mapping runs first.
3. **Template migration** — baseline `SYSTEM_SETUP` values the file is missing are added. Existing values are never overwritten.
4. **Building code normalization** — a `gve_bldg` holding a full building name is converted to its code.
5. **Device defaults fill** — device blocks missing their family's baseline properties get them (switch this off in App Config if you'd rather it didn't).
6. **Auto room name** — `gui_full_room_name` is rebuilt in Title Case.
7. **Audits** — more device blocks than the count allows, or keys that aren't in the template, are reported. Nothing is changed.

Everything the pipeline did shows up in the dialog afterwards, in the migration log, and in a change log written next to the backup. If you don't like what it did, the pristine original is right there.

Getting the config back out

- **Export Config Locally** — a save dialog, named from the building and room (`BSS_103_config.json`).
- **Upload to Processor (SFTP)** — pushes to `/config.json`, optionally with a companion file such as `Whereused.csv`. The dialog has the same searchable room picker as the download.
- **Apply Changes** on the Raw JSON tab — parses the text back into memory *and* writes the working file, so Apply doubles as Save.

Every path out writes keys sorted in natural order, so `PROJECTORDEVICE_2` comes before `PROJECTORDEVICE_10` and two exports of the same room diff cleanly.

Devices

Each device gets a sub-tab: its model, how the processor reaches it, and the python module that drives it. *Check Defaults* tells you what a block is missing; *Check Module Defaults* tells you where it disagrees with the driver.

When the model and the driver don't match

A device block names a product and names the driver that talks to it, and nothing used to keep the two together. Retype the model, or swap the box from the **Cost** tab, and the module underneath went on naming a driver for the product that used to be there — with every field filled in, so the block read as finished. That's the config nobody re-checks, and it commissions the room as a device the room doesn't contain.

A red banner now sits at the top of the device, above the two fields that fix it, in two cases:

- **no python module is set** for a model that has one — nothing can talk to the device, so the room won't commission
- **the module is set, says which models it covers, and this isn't one of them** — the banner names what that driver actually drives

Pick a module (or correct the model) and it clears. It's read from the config rather than remembered by the page, so it survives leaving the tab, saving, and reopening the room — and it's what a swap made on the Cost tab leaves behind.

It stays quiet where it can't know: a device with no model yet, a driver that never declared which models it covers, and a model the driver does list under a different capitalisation.

The drawings

Control Schematic

This one answers a single question: *what does the processor talk to, and how?*

It draws itself from the config every time you open it. Network devices go to a Network IDF box and up to the processor; serial devices get a direct line labelled with their COM port; relay-controlled screens get a relay line; the touch panel is drawn as a window with one tab per GUI page. Each kind of connection has its own colour, and the legend under the drawing explains them.

Things worth knowing:

- **192.x drops** aren't on the building network — they're on the AV LAN. The toolbar has a dropdown for where those actually land in your buildings: the IDF, the processor's own second NIC, or a switch you draw yourself. The touch panel has its own dropdown, because a panel on PoE off the building switch is as ordinary as one beside the processor.
- **Add device** (in Edit mode) puts a box on the drawing for equipment the control system doesn't talk to at all — the building switch, a UPS, the room PC. They're drawn as related equipment, dashed rather than solid, so nobody reads them as controlled devices.
- **Save Layout** keeps your dragging next to the config; **Reset Layout** puts every box back where the auto-layout wants it.

Recreate from config

Sometimes the drawing has drifted far enough that tidying it costs more than starting again — a room got re-scoped, half the devices changed, and there are hand-drawn lines to boxes that no longer exist.

Recreate from config throws the layout away and lets the drawing rebuild itself: every box back at its automatic spot, every generated line back including any you had hidden, and hand-added boxes removed. It asks first and tells you exactly what it will take with it, and one press of Undo brings it all back.

Two things it deliberately keeps: your line **colours** (they have their own *Reset all* on the Colors dialog) and the **192.x / touch panel landing** choices, because where the room's drops actually go is a fact somebody recorded, not drift. The one exception is a landing pointing at a hand-added box that's being removed — that goes back to the IDF, since the drops would otherwise land on nothing.

AV Flow

The AV Flow is the signal flow: sources, the switcher, displays, and every lead between them.

It draws itself too, from two things you have already typed. The **device blocks** say what the room has. The **input and output numbers** in `SYSTEM_SETUP` say what plugs in where: `input_pc: "1"` means the room PC is on switcher input 1, `output_proj_1: "3B"` means display 1 is on output 3's B connector. The app places the

boxes, finds the right connector on each end, and draws the lead.

It also puts in the boxes nobody writes down because everybody knows they're there:

- A **DTP receiver** behind a display whose matrix output is twisted pair and whose socket is HDMI. That run is two cables and a box, and a drawing that joins them directly is drawing a cable that can't be bought — and an estimate taken off it is missing a $600 box per display.
- A **DTP transmitter** beside a camera on a DTP input, which is the same fact read the other way round.
- The **expansion bus** between a switcher with a DSP in it and the DMP racked beside it, rather than an analog lead into a socket nobody patches.
- The **USB chain** into a USB switcher — see below.
- **Power leads** from the controller, worked out from the outlet names you typed on the System tab. Outlet 1 says "PC", so it goes to the PC.

Every one of those is a rule you can change on the **Flow Rules** tab. Nothing in this list is compiled in.

The USB switcher

Nothing in the config says what plugs into a USB switcher — `dev_usb_switchers` is a count, and there's no input number for a USB lead anywhere in the file. So the drawing used to show a Toggle with nothing plugged into it, and the conferencing path stopped at the DSP.

It's now drawn from the way these rooms are actually wired:

Connector	What lands on it
USB DEVICE 1	the DSP's USB output — the microphone mix
USB DEVICE 2	the AV Bridge's USB output — the room's picture
USB DEVICE 3	the document camera's USB output
USB HOST 1	the room PC

Those are fixed positions, not a queue. A room with no doc cam leaves DEVICE 3 empty rather than sliding the AV Bridge onto it, because the port a lead lands on is what the tech reads off the drawing. Anything already plugged in by hand is left exactly as you left it. And if your rooms are wired differently, that table is four lines on the Flow Rules tab.

Working on the drawing

- **Edit** mode lets you drag boxes, draw cables between ports, add bends to a run, and open any box to correct its connectors.
- **Place all from config** adds every config device that isn't on the canvas — including any you deleted earlier.

- **Draw the routing from config** shows you what the config's numbers would draw, tie by tie, with a reason for anything that didn't resolve, before it draws it.
- **Recreate from config** starts over: every box and every cable removed, then the room drawn again from the config and the current rules. It asks first, it is one press of Undo, and it keeps the rack rails of devices that come back — re-reading the config is no reason to unrack the room.
- **Room type presets** stamp a whole standard room onto the canvas, wiring and all, for the builds you do over and over.

Reading the routing report

Some ties can't be drawn, and the app would rather tell you than guess. Press *Draw the routing from config* and you get a line per tie: what the config said, what it resolved to, and — for the ones that didn't — why. Typical reasons:

- *"No input on the switcher is labelled 7"* — the number doesn't match any connector on that model. Usually the catalog entry needs its connectors correcting.
- *"output_proj_2 — this room has 1 display, so there is no PROJECTORDEVICE_2"* — dead config left over from when the room was bigger.
- *"Every input on the recorder that could take 2 is already fed"* — the box has one socket and the config asks for two feeds.

Racks, plans, cabling and money

Racks

Drag anything on the drawing into a frame and it takes the rails its catalog entry says it needs. The elevation shows front and rear, shades the rails a model wants left clear (the amp that vents upward, the drawer whose lid opens), and warns rather than refuses — the person in front of the frame knows things the catalog doesn't.

Rack hardware that isn't a device — vent plates, shelves, drawers, blanks — is added from the parts list and counts on the estimate like anything else.

Floor Plan

A sheet per room view, with callouts pointing at named places: Instructor station, Front wall, Ceiling, Equipment rack. Every device on the AV Flow can be given a location, and the callouts, the reports and the cable schedule all use the same names.

Cabling

The low-voltage runs that aren't signal flow: screen triggers, shade control, the runs between places in the room. It produces a run schedule with lengths, so somebody can pull cable off it.

Cost

The estimate reads the drawing, not a list you maintain by hand. Every box on the AV Flow is priced from the catalog; every cable is priced by length; labor comes from the rate card, tax and fees from the settings on the tab.

Two things it will tell you rather than quietly getting wrong:

- devices on the drawing whose model the catalog doesn't price, counted so you know how much of the estimate is guesswork
- devices with no control module, so a room isn't quoted as finished when part of it can't be driven

Is it in the room config?

The estimate is where a room gets specified — parts picked with quantities and a total — and the control side is usually built weeks later. A line that never becomes a device block is a box that gets ordered, delivered, racked, and then has nothing to drive it.

So every equipment row carries a flag:

- **orange flag** — quoted, and the room config has never heard of it. The flag is also the button: *Add to the room config* creates the device block for its family with this room's defaults and the driver that claims the

model. A line typed here becomes boxes on the diagram first (one per unit), because a device has to exist before it can have a block — so the cost line is replaced by the drawn devices it was quoting.

- **a tick** — it's in the config, and the tooltip names the block.
- **a box icon** — marked as a **spare**: bought for the shelf on purpose, never installed here at all. It stays on the quote and stops being flagged.
- **a broken-link icon** — marked **not part of the room config**: it *is* in the room and on the diagram, and the processor has no business talking to it. The building's network switch, a codec another department manages, an owner-furnished display, a passive splitter. It stays exactly as it was — drawn, selectable, cabled, racked and priced — and drops off every "missing from the config" list: the flag here, the control-side prefill, and the missing-module report. The same checkbox is on the box's own editor on **Signal Flow**, next to *Not on the cost estimate*, and the box wears a small broken-link badge on the diagram.

Those last two are the escape hatches that keep the flag meaningful. Every room has boxes whose honest answer to "why is this not in the config" is "it never will be", and a warning nobody can clear is a warning everybody learns to ignore.

A hand-typed line with no catalog part behind it can't go in — there's nothing to build a device from. Add it to the catalog first with the library button on the same row.

Base costs for cable

Cable is the hole in every early estimate: the runs are counted off the diagram exactly, then priced at nothing, because the catalog ships made-up leads for a handful of types while a real order is "a 25 ft HDMI and a 50 ft HDMI".

The **price tag** button on a cabling row puts the shop's typical figure for that type and length on the base-cost card — `base_costs.json`, the same file the device figures live in, read by every room. Enter it once and every estimate prices that length off it. A price typed on the room still wins, and a line costed this way is marked *Base cost* and counts towards the estimate being a budget rather than a quote.

A figure entered against a type with no length ("Cable: HDMI") covers every length that hasn't got one of its own.

Swapping a unit

The wrong box usually gets noticed here — the total is what people look at — so the fix is here too. The **replace** button on the right of an equipment row picks the new product out of the catalog and puts it everywhere the room records the old one:

- **the cost line** reprices off the new catalog entry
- **the signal flow** gets the new product under every box the line counts — connectors, rack height, power and heat — and the runs already drawn move onto the matching connectors
- **the cabling schematic and cable schedule** follow, because they're built from the flow rather than stored

- **the control side** — the config block the box came from — gets the model and the Python module that claims it
- **the name**, where the name was the product: "Projector 1 - PowerLite L630U" becomes "Projector 1 - PT-MZ682BU8". Only the model part moves — "Projector 1
    - " is what the room calls that position, and the position hasn't changed — and a name that never mentioned the model is left alone

A line of quantity three swaps all three boxes and all three config blocks. Any price you had typed against the old model is cleared, because a figure negotiated for one product isn't the price of another.

It works in both directions. Picking a model on the **Devices** tab moves the same room: the drawn box becomes that product — connectors and all, where the AV catalog knows the model — the name follows, and the estimate reprices, because the estimate counts the boxes on the diagram.

It goes through silently when there's nothing to decide. It stops to ask when there is:

- **no module claims the new model.** You can still go ahead — a part often arrives before its driver does — but the module is cleared off the config block rather than left naming a driver for the old box, and the device shows a red banner on the **Devices** tab until somebody picks one. See *When the model and the driver don't match*.
- **the module changes and this room's settings disagree with its defaults.** The same question the Devices tab asks when you pick a model there, with the same two answers: keep the room's settings (the IP address and port are facts about the install) or apply the new module's defaults.

Cancel means nothing happened — you're asked before the first write. A run the new model has no connector for is removed rather than left pointing at nothing, and the message says how many, so you can draw them again on **Signal Flow**.

Catalog

The catalog is the department's price list and connector reference: per model, what sockets it has, how many rack units, what it draws, what it costs.

Searching ignores spaces, dashes and case on both sides, so "dtpcross108" finds "DTP CrossPoint 108". Type more than one word and every word has to land somewhere on the entry — the model, the maker, the part number or the category — in any order: "Epson PowerLite" finds the PowerLite projectors, even though no single field holds those two words next to each other. Adding words still narrows the list.

The room config knows a device's IP address; it never knows that the box has four HDMI inputs, is 2U, draws 90 W and lists at $8,500. That's what the catalog is for, and it's why one shared copy is worth keeping.

Merge exists because two engineers keep two copies and neither is authoritative: one has priced the switchers, the other has drawn their connectors. Point it at their file and every difference comes up with its own checkbox.

Getting work out of the app

Save All writes the whole room into one folder: every drawing as a PNG, every report, the workbook, and the config itself. It walks the drawing tabs to capture them, then puts you back where you started.

The room workbook is one `.xlsx` with a sheet per document — devices, control, AV, racks, cabling, cost — for handing to somebody who doesn't have the app.

Per-tab exports are on every tab's own menu: this page's tables as `.xlsx`, as plain text, or straight to the clipboard.

Screenshot & annotate grabs the current tab, lets you draw on it, and copies or saves it — for the email that starts "the third output is wrong".

The Flow Rules builder

What a rule is

Drawing a room takes two kinds of knowledge, and only one of them belongs to the program.

The mechanical half is the app's: which socket the number `3B` names on a DTP CrossPoint, whether a connector already has a lead on it, how to split a run into two cables and a box. You never have to think about that.

The other half is a set of decisions *your shop* has made:

- `input_pc` means the room PC, and the room PC is a box with an HDMI out and a USB in
- a twisted-pair output reaching an HDMI display needs a DTP 4K 230 receiver between them
- the Toggle's DEVICE ports carry the DSP, the AV Bridge and the doc cam, in that order, and HOST 1 is the PC
- an outlet labelled "Switch" is the matrix, not the USB switcher

Every one of those used to be compiled into the program. Now they're rules in `av_flow_rules.json`, and the **Flow Rules** tab is where you read and change them.

> With no rule file at all, the app uses its built-in rules — the same ones that used to be in the code. Nothing you have to do; the tab is there for when the defaults are wrong for a room, a building or a whole campus.

Finding your way around the tab

The list on the left is the rule families, with a count each. Pick one and the right-hand side shows what's in it, one card per rule, with an **Add** button above and edit and delete buttons on each card.

Along the top: **Save** writes the file, **Reload from file** throws your edits away and re-reads it, and **Reset to built-in** puts back exactly what the app ships with. Next to the title you'll see either the file the rules came from or *"Edited — not saved yet"*.

Edits take effect immediately, before you save. The file on disk is only touched by Save, so you can try a rule, look at a room, and decide.

> To see a rule change on a room that's already drawn, go to the **AV Flow** tab and press **Recreate from config**. The drawing is rebuilt from the config and the rules as they now stand.

Naming a box in a rule

Several rules have to point at "whatever box in this room is the DSP". One small syntax does that everywhere:

You type	It means
`DSPDEVICE_`	the family — the first numbered block of it that fits
`RECORDERDEVICE_1`	exactly that config block
`input_doc_cam`	the box the source rules place for that config key
`AV Bridge 2x1`	a catalog model, matched against what's on the canvas
`RECORDERDEVICE_|MEDIAPORTDEVICE_`	try the first, then the second

The family form is the one you want most of the time. "The DSP" means the DSP this room has, whether it's block 1 or block 3, and the app skips blocks that can't do the job — a DSP with no USB socket isn't the DSP the USB rule means.

The families, one by one

Source boxes

An `input_` key whose box has no config block of its own: the room PC, the doc cam, the laptop at a plate, a DVD player. The config says which switcher input it's on; this rule says what the box actually is.

Each one carries a name for the drawing, a catalog model (where its connectors, price and rack height come from), where in the room it lives, and whether the room is paying for it — the presenter's own laptop belongs on the drawing but not on the quote.

Source devices

An `input_` key naming a device the config already describes, so the box is already on the canvas and only the cable is missing: `input_wireless` is the wireless block, `input_inst_cam` is camera 1.

Display outputs

The other end of the matrix: `output_proj_1` feeds `PROJECTORDEVICE_1`. If a room has a display output for a display it doesn't have, you get a plain-English finding rather than a mystery gap.

Destination boxes

Same idea as source boxes, at the other end: a confidence monitor, the assisted-listening transmitter. These carry one extra setting — which connectors the lead may land on. Assisted listening is line audio, and looking for a video socket on it finds nothing, which is why the rule says so rather than the program special-casing the key by name.

Capture

Where the capture feed lands. A room's capture box is a MediaPort in one build and an AV Bridge in another, so the rule names the alternatives in the order they should be looked for.

Extenders

The most useful family, and the one worth understanding.

A rule here says: *when a run leaves the switcher on one kind of connector and arrives at the far end on another, this is the box that goes between them.* The run becomes two cables and a box on the drawing, and the box lands on the estimate.

The shipped rules are the two directions of the same fact:

At the switcher	At the far end	Which end	Box
`hdbaset`	`hdmi`	an output	DTP HDMI 4K 230 Rx
`hdbaset`	`hdmi`	an input	DTP HDMI 4K 230 Tx

Before it invents anything, the app checks whether the far end takes twisted pair itself — a projector with an HDBaseT socket needs no receiver, and putting one in quotes a box the room doesn't need.

USB switchers

One line per port, in port order: the first line is DEVICE 1, the second DEVICE 2, and so on, and the same for the HOST ports. Leave a line blank to leave that port empty.

The shipped rule for a Toggle reads:

```
The USB switcher:  USBDEVICE_1

DEVICE ports       DSPDEVICE_
                   RECORDERDEVICE_|MEDIAPORTDEVICE_
                   input_doc_cam

HOST ports         input_pc
```

Expansion bus

The words that identify an expansion-bus connector on a box — `EXP` and `EXPANSION` as shipped. Matched as whole words, so `EXP` doesn't also match `EXPO`. If a manufacturer spells theirs differently, add the word.

Outlet names

Outlet labels that name a device outright, whatever else on the canvas answers to the word. "Switch" is the one that needs saying: in a room built out of Extron gear it means the matrix, but scored on the word alone it ties with the USB switcher and every "Switcher 2" — and a tie draws nothing at all.

Matched on the whole label, so this settles "Switch" and leaves "USB Switch" alone.

Five things you might actually want to do

The room has a kind of source the app doesn't know

Say your template gained `input_cable_box`.

1. Go to **Source boxes** and press **Add**.
2. Config key: `input_cable_box`.
3. Name on the drawing: `Cable TV box`. Catalog model: the model from the Catalog tab. Where it lives: `rack`.
4. Save the rule, then **Recreate from config** on the AV Flow tab.

The box is placed and cabled to whatever input the config gives it. If the catalog doesn't carry that model yet, the card tells you so — the box still draws, with the generic family connectors, and costs nothing on the estimate until you add it.

We use a different receiver

Open **Extenders**, edit the receiver rule, and change the model to the one you actually buy — `DTP HDMI 4K 330 Rx` for the long runs, say. Every room you redraw from then on puts that box in, on the drawing and on the quote.

Our Toggles are wired the other way round

Open **USB switchers**, edit the rule, and reorder the DEVICE port lines. If your doc cam is on DEVICE 2 and the AV Bridge on DEVICE 3, swap those two lines. That's the whole change.

This building's capture box is a MediaPort

Nothing to do — the shipped capture rule already tries a MediaPort, then a recorder, then a USB switcher. If you have a fourth kind of box, add it to the end of the list with a `|`.

An outlet label your team uses

Open **Outlet names** and add it: the whole label on the left, the device family it means on the right. Now that outlet gets a mains lead drawn to the right box instead of being reported as a tie.

When a rule doesn't fire

- **The card shows a red line about the catalog.** The model named in the rule isn't in `av_devices.json`. Not fatal — the box draws with the generic connectors for its family — but it's nearly always a typo, and it will cost nothing on the estimate.
- **Nothing is drawn for a key.** Check the config actually has that key with a value. A blank or `none` means "there is no cable", and that's respected.

- **A box you named isn't found.** Family prefixes need the trailing underscore (`DSPDEVICE_`, not `DSPDEVICE`). A model name has to match the catalog spelling.
- **The drawing didn't change.** Press **Recreate from config** on the AV Flow tab. Rules apply to what gets drawn *next*; they don't rewrite leads already on the canvas.
- **You want the old behaviour back.** *Reset to built-in* returns every family to the shipped rules. Nothing is written until you press Save.

The Schema builder

What the schema decides

Open the **Devices** or **System** tab and every field you see — its label, the description behind the info button, whether it's a switch or a dropdown, what that dropdown offers, whether it appears at all — comes from `ui_schema.json`. So does the list of device families the Wizard manages, and what a new or newly loaded room is given when it's missing something.

That file has always been editable. What was missing was a way to edit it *here*, against a real config, instead of in a text editor with a second window open to see which keys you'd covered.

Coverage: the part you'll use most

Coverage is that second window, built in.

Pick a block of your default config file — `SYSTEM_SETUP`, `PROJECTORDEVICE_1`, whatever the template has — and every key in it is listed against the schema entry that describes it, with a count at the top: *"38 of 120 keys described"*. Turn on **Not described yet** and you're looking at the list of fields that currently show up as a raw key with a plain text box.

Press **Describe** beside one and the field editor opens with the key already filled in and its type guessed from what the file holds — a `true` becomes a switch, a number becomes a numeric field. Give it a label and a description, press Save, and go and look at the System tab: it's already there.

Another config file points Coverage at a different config, which is how you check the schema against a real room rather than the template.

Describing a field

The field editor holds everything a schema entry can say:

Setting	What it does
Config key	The key it describes. May contain a `*` to cover a family: `power1_outlet_*`.
Rendered as	The control: `auto`, `text`, `int`, `double`, `bool`, `dropdown`, `combo`, `hidden`, `room_sources`, `module_states`.
Label	What the field is called on the tab.
Description	The text behind the info button.
Helper line	Small grey text under the field.
Options	For a dropdown: one per line, `value | label`. The label is optional.

Setting	What it does
Keys this one field writes	For a combo — one dropdown that sets several keys at once.
Command in the module	For `module_states`: the entry in the driver's `self.Commands` whose states fill the dropdown.
Hide when	Conditions that make the key irrelevant. Any one true and the field isn't drawn, isn't added to new devices, and isn't offered by Check Defaults.
Label when	A different label under a condition, first match wins.
Show even when the block has no such key	The field appears anyway, and the first edit writes it. This is how a new setting reaches rooms built before it existed.

Conditions are written the same way everywhere: `key=value`, `key!=value`, or `key~text` for "contains". All case-insensitive.

The other sections

Fields is the plain list of everything the document describes globally, with a search box — the same editor, reached from the other direction.

Device fields are entries scoped to one family or section, keyed by a pattern like `PROJECTORDEVICE_*`. A scoped entry wins over a global one on those tabs, which is how a projector's `input` can be a dropdown filled from its own driver while `input` elsewhere stays plain text.

Device families is the Devices tab itself: each family is a `dev_` count key, the section prefix its blocks use, a label, the highest count the Wizard offers, and any `SYSTEM_SETUP` keys the family owns (setting the count to 0 removes those, because outlet names with no power controller behind them are dead data). Add a family here and it gets Wizard counts, device tabs, pruning, audits and a `"0"` default with no code change at all.

> Editing any family writes the *whole* list into the file. That's deliberate: defining families at all replaces the built-in list, so a half-list would silently drop the rest. The tab tells you when it's about to do this.

Defaults covers the three kinds:

- `SYSTEM_SETUP` defaults, added to a loaded room that's missing them
- whole **section blocks** a room is given when it has none (a metrics block, say)
- **device defaults**, merged into every new block of a family — projectors get their `input` and `relay_host`, DSPs get their audio group numbers

Each is edited as `key = value` lines, one per line, with JSON values so `true`, `3` and `"text"` all keep their type.

Consistency is the cross-check: *when this is true, that must be true too*. A violation never blocks an edit — it paints the red mismatch outline on the fields the rule names, with your message as the red helper line. `{key}`

in the message is replaced with that key's live value.

Raw JSON is the whole document in a text box, validated before it's applied. Anything the forms don't cover lives here, and a document that won't parse changes nothing.

Saving, and what's kept

Save writes the document that was **read**, with your edits in it — not a regenerated approximation. Two consequences worth having:

- the `__comment` entries the file uses to explain itself survive
- so does anything a later build understands and this one doesn't

If nothing has ever been read — you're running on the app's built-in schema — Save refuses, rather than replacing a schema somebody has with an empty document. Point App Config at a `ui_schema.json`, or start one from Raw JSON.

Four things you might actually want to do

The template gained a key and the field looks raw

Coverage → the block → **Not described yet** → **Describe**. Label, description, type. Save.

Make a text box into a dropdown

Edit the field, set **Rendered as** to `dropdown`, and put the options in one per line. Use `value | label` when the stored value isn't what you want people to read:

```
Yes | Yes (single mic with ducking)
No  | No (single mic with mute)
Ceiling | Ceiling (voicelift and mute)
```

Hide a field that doesn't apply

A serial port means nothing on a device connected over the network. Put `com_type=Network` in **Hide when** and it stops being drawn, stops being added to new devices, and stops being offered by Check Defaults — and the keys that just became irrelevant are removed when you change the gating value.

Add a whole new device family

Device families → **Add family**. Count key `dev_lifts`, prefix `LIFTDEVICE_`, label `Projector Lifts`. Save, then look at the Wizard: it has a count dropdown for lifts, and setting it to 2 creates the blocks and their tabs. Give the family its fields under **Device fields** with the pattern `LIFTDEVICE_*`, and its baseline values under **Defaults**.

Reference: ui_schema.json

Everything here can be done on the **Schema** tab. This chapter is for when you would rather look at the file — reviewing a change, or diffing two copies.

The shape of the file

One JSON object. `fields` is the only part it must have; everything else is optional, and a part you leave out keeps the app's built-in behaviour.

```
{
  "schema_version": 1,
  "__readme": [ "keys starting with __ are ignored, so notes live here" ],

  "fields":            { "com_type": { ... } },
  "device_fields":     { "PROJECTORDEVICE_*": { "input": { ... } } },
  "section_fields":    { "METRICS_CONFIG":    { "enabled": { ... } } },
  "device_defaults":   { "DSPDEVICE_*": { "group_prog_gain": "1" } },
  "device_types":      { "dev_projectors": { "prefix": "PROJECTORDEVICE_" } },
  "system_defaults":   { "gui_mic_mix": "No" },
  "section_defaults":  { "METRICS_CONFIG": { "enabled": false } },
  "consistency":       [ { "when": "...", "expect": "..." } ],
  "runtime_written":   [ "SYSTEM_SETUP.last_boot" ]
}
```

A field entry

```
"com_type": {
  "type": "dropdown",
  "label": "Connection Type",
  "description": "Text behind the (i) info button.",
  "helperText": "Small grey hint under the field.",
  "options": ["Serial", "SerialOverEthernet", "Network"],
  "hideWhen": ["dev_relay_power=No"],
  "labelWhen": { "gui_usb_or_vga=VGA": "VGA over USB" },
  "addIfMissing": true
}
```

Property	Means
`type`	Which control to draw. See the table below.
`label`	The field's name on the tab. Falls back to the raw key.

Property	Means
`description`	The info button's text. Falls back to the app's built-in dictionary.
`helperText`	Grey hint under the field.
`options`	Dropdown or combo choices.
`writes`	Combo only: the keys this one dropdown sets.
`moduleCommand`	`module_states` only: which command's states to offer.
`hideWhen`	Conditions that make the key irrelevant to its block.
`labelWhen`	Condition to label, first match wins.
`addIfMissing`	Draw it even when the block has no such key yet.

Types

Type	Draws
`auto`	Inferred from the value: bool to a switch, int to a numeric field, anything else to text.
`text`, `int`, `double`	A text field that stores that type.
`bool`	An on/off switch.
`dropdown`	A fixed list of options.
`combo`	One dropdown that writes several keys together.
`hidden`	Never drawn — for keys a combo or the Wizard manages.
`room_sources`	A dropdown of the sources *this* room has, read off its `input_*` keys.
`module_states`	A dropdown filled from the device's own Python module.

Options, two ways

Plain strings when the stored value is what you want to read:

```
"options": ["TCP", "SSH", "UDP"]
```

Objects when it isn't:

```
"options": [
  { "value": "Yes",     "label": "Yes (single mic with ducking)" },
  { "value": "No",      "label": "No (single mic with mute)" },
  { "value": "Ceiling", "label": "Ceiling (voicelift and mute)" }
```

```
]
```

Combos

One dropdown, several keys. Name the keys in `writes` and give each option a `values` array in the same order:

```
"gui_inputs": {
  "type": "combo",
  "label": "Sources & Routing",
  "writes": ["gui_inputs", "gui_tab_type"],
  "options": [
    { "label": "PC + HDMI + Doc Cam", "values": ["3", "DOC_USB_WL"] },
    { "label": "PC + HDMI",           "values": ["2", "USB_WL"] }
  ]
}
```

Wildcards

A `*` in the key covers a family: `power1_outlet_*` describes every outlet name in one entry. An exact key always wins over a wildcard.

Device families

```
"device_types": {
  "dev_projectors": { "prefix": "PROJECTORDEVICE_", "label": "Projectors / Displays" },
  "dev_lifts":      { "prefix": "LIFTDEVICE_", "label": "Projector Lifts", "max": 4 },
  "dev_power_controllers": {
    "prefix": "POWERDEVICE_",
    "label": "Power Controllers",
    "systemKeys": ["power*_outlet_*"]
  }
}
```

`prefix` is required; everything else is optional. `max` is the highest count the Wizard offers (8 by default), `keepAlivePreference` is the command names tried in order when a new device picks a keep-alive off its module, `template` is a whole block to use for new devices when the config has no block 1 to copy, and `systemKeys` are the `SYSTEM_SETUP` keys that mean nothing without this hardware — setting the family's count to 0 removes them.

> Defining `device_types` at all replaces the built-in list. Keep every family you want, in the order you want them in the Wizard. The Schema tab does this for you.

Defaults

- `system_defaults` — `SYSTEM_SETUP` values injected into a loaded room that is missing them. Defining any replaces the built-in set. The `dev_` counts are not listed here; they always come from `device_types` so the two can't disagree.
- `section_defaults` — whole blocks a room is given when it has none.
- `device_defaults` — merged into every newly created device block. Existing values are never overwritten, and an exact section name beats a wildcard.

Consistency rules

```
"consistency": [
  {
    "section": "SYSTEM_SETUP",
    "when":   "gui_tab_type~VGA",
    "expect": "gui_usb_or_vga=VGA",
    "message": "This tab type shows a VGA source, but gui_usb_or_vga is {gui_usb_or_vga}.",
    "flag": ["gui_tab_type", "gui_usb_or_vga"]
  }
]
```

Runtime-written keys

`runtime_written` lists `SECTION` or `SECTION.key` entries (with `*` wildcards) that the *processor* writes while the room runs. They stop a downloaded config being flagged for carrying things the template never had.

Reference: av_flow_rules.json

Everything here is edited on the **Flow Rules** tab. A family the file leaves out keeps the app's built-in rules; a family it defines replaces them outright.

```
{
  "sourceBoxes": {
    "input_pc":       { "label": "Room PC", "model": "PC" },
    "input_doc_cam":  { "label": "Document camera", "model": "Document Camera" },
    "input_hdmi":     { "label": "Laptop at the HDMI plate", "model": "HDMI Laptop",
                        "excludeFromCost": true },
    "input_dvd":      { "label": "DVD player", "model": "DVD Player", "zone": "rack" }
  },

  "sourceDevices": {
    "input_wireless": "WIRELESSDEVICE_1",
    "input_inst_cam": "CAMERADEVICE_1"
  },

  "destinationDevices": {
    "output_proj_1": "PROJECTORDEVICE_1"
  },

  "destinationBoxes": {
    "output_monitor_1": { "label": "Confidence monitor", "model": "Confidence Monitor" },
    "output_audio_ald": { "label": "Assisted listening", "model": "Assisted Listening",
                          "zone": "rack", "signals": "lineAudio" }
  },

  "captureDestinations": {
    "output_cc": "MEDIAPORTDEVICE_1|RECORDERDEVICE_1|USBDEVICE_1"
  },

  "extenders": {
    "rx": { "switcherSignal": "hdbaset", "farSignal": "hdmi", "onOutput": true,
            "model": "DTP HDMI 4K 230 Rx", "label": "Room-end DTP receiver",
            "zone": "wall" },
    "tx": { "switcherSignal": "hdbaset", "farSignal": "hdmi", "onOutput": false,
            "model": "DTP HDMI 4K 230 Tx", "label": "DTP transmitter",
            "zone": "lectern" }
  },

  "usbSwitchers": [
```

```
    { "switcher": "USBDEVICE_1",
      "devicePorts": ["DSPDEVICE_", "RECORDERDEVICE_|MEDIAPORTDEVICE_", "input_doc_cam"],
      "hostPorts":   ["input_pc"] }
  ],

  "expansionKeywords": ["EXP", "EXPANSION"],
  "outletAliases":     { "switch": "SWITCHERDEVICE_" }
}
```

Field	Means
`label`	What the box is called on the drawing.
`model`	Catalog model — connectors, rack units, price.
`zone`	`lectern`, `rack` or `wall`.
`excludeFromCost`	Draw it, don't quote it.
`signals`	Which connectors a lead may land on: `video`, `lineAudio`, `speaker`, `usb`.
`switcherSignal` / `farSignal`	The two ends that don't match.
`onOutput`	`true` for a receiver at a display, `false` for a transmitter at a source.
`devicePorts` / `hostPorts`	In port order. An empty string leaves that port free.

The laptop plate is one config key with two meanings, so the rule book carries both boxes: `input_usb` for a USB-C room and `input_usb (VGA room)` for a VGA one. `gui_usb_or_vga` picks between them.

Reference: key_map.json

The key map translates old files. It runs at the very start of every load, turning legacy names into current ones before anything else sees the config. With no file present, nothing is translated and loading behaves as it always did.

The order it works in

Each step depends on the ones before it — moves target already-renamed sections, companions see converted key names:

1. `sections` — rename top-level blocks
2. `properties` — explicit per-property renames
3. `auto_case_normalization` — automatic case and underscore fixes
4. `value_map` — normalize stored values
5. `moves` — relocate a property into another section
6. `remove_unused_devices` — drop legacy blocks not in use
7. `escape_carriage_returns` — turn control characters into the file's `\r` marker
8. `device_labels` / `device_templates` — friendly names, buttons, labels, modules
9. `companion_keys` — create partner keys
10. `defaults` — inject anything still missing

The rule groups

sections — a list of `{ match, rename_to }`, where `match` is a whole-name regex and `$1`..`$9` insert capture groups. First match wins.

```
"sections": [
  { "match": "CAMERA(\\d+)DEVICE", "rename_to": "CAMERADEVICE_$1" },
  { "match": "CAMERADEVICE",       "rename_to": "CAMERADEVICE_1" },
  { "match": "SYSTEM|SYSTEMSETUP", "rename_to": "SYSTEM_SETUP" }
]
```

properties — a flat old-name to new-name map applied inside every section. Always wins over automatic normalization.

auto_case_normalization — when true, a legacy property whose lowercased, underscore-stripped form matches a known current key is renamed automatically, so `Output_Proj1`, `GVE_BLDG` and `COMTYPE` need no entries of their own. Only spellings that differ structurally need one.

value_map — keyed by the *new* property name. Mapped values keep their JSON type, so a string can become a boolean.

```
"value_map": {
  "active_notifications": { "True": true, "False": false },
  "com_type": { "NETWORK": "Network", "SERIAL": "Serial" }
}
```

moves — relocate a property into another section, capture groups usable in the target name. Legacy files kept per-device GVE IDs in `SYSTEM`; this is how they reach each device.

remove_unused_devices + **device_counts** — legacy blocks whose family count says they aren't in use are removed rather than converted. Only blocks that were *renamed* in step 1 are eligible, so a current-style template that deliberately carries every block keeps them all.

escape_carriage_returns — real control characters inside strings become the two-character marker the processor reads, so an outlet name arrives as `"Doc\\rCam"` and breaks over two lines on the panel button.

companion_keys — guarantee a partner key exists for everything matching a pattern, with the number carried over. Every `power1_outlet_N` gets a `power1_outlet_N_action`, only for outlets that exist, and an existing value is never overwritten.

device_labels / **device_templates** — friendly fill-in when converting: `name` built as "Label - model", numbered only when the family has more than one unit; `btn_name` and `lbl_name` carried over by device number; `module` and `keep_alive_trigger` carried over when the legacy model matches a template block.

defaults — after everything else, any listed key still missing is injected. `{n}` in a value becomes the device number.

When something looks wrong

What you see	What it usually is
A field shows a raw key name and a plain text box	No schema entry for it. **Schema → Coverage → Describe**.
A red outline on two fields	A consistency rule says they disagree. The red helper line says which.
A device tab you didn't expect, or one missing	The family's `dev_` count on the Wizard, or the family list under **Schema → Device families**.
The AV Flow is missing a lead	Press **Draw the routing from config** and read the reason. Usually a blank config number or a connector the catalog doesn't list.
A run drawn straight from a DTP output into an HDMI socket	The extender rule for that pair was removed. **Flow Rules → Extenders**.
Nothing plugged into the USB switcher	The USB rule names boxes this room doesn't have, or their catalog entries have no USB connector.
An outlet's mains lead isn't drawn	Two boxes answered to the name equally well, so nothing was drawn. Add an entry under **Flow Rules → Outlet names**.
A box on the drawing that costs nothing	Its model isn't in the catalog. The Cost tab counts these for you.
A drawing that no longer matches the room	**Recreate from config**, on either the AV Flow or the Schematic tab.
The wrong room in the SFTP dialog	The active deployment target clears when you open a different config — pick the room again.

Which file does what

File	Read when	Written by	If it's missing
`config.json` (template)	New Config	Never	New Config can't run
`ui_schema.json`	Startup, Reload Schema	Schema tab	Built-in field definitions
`key_map.json`	Every load	Text editor	Nothing is translated
`av_devices.json`	Startup, Reload Catalog	Catalog tab	Built-in models, no prices
`av_flow_rules.json`	Startup, Reload Rules	Flow Rules tab	Built-in drawing rules
`processors.json`	Startup	Text editor	No room list in the SFTP dialogs
`buildings.json`	Startup	Text editor	No building names or codes
`labor_rates.json`	Startup	Cost tab	Built-in roles, no rates
`base_costs.json`	Startup	Cost tab	Every category unpriced
`app_config.json`	Startup	App Config tab	First-run setup runs

Keeping this guide honest

This document is built from `documentation/Room_Config_Builder_Guide.md` in the repository. Edit that file and run:

```
python tools/build_guide.py
```

which rewrites both `documentation/Room_Config_Builder_Guide.pdf` and `Room_Config_Builder_Guide.docx`. Keeping the source in the repo means a change to the app and the change to its guide can travel together, and anybody can see what moved.